

Project Proposal

PeraMorphIQ: Autonomous DOOM Gameplay

Software Pipeline via Spiking Neural Networks (SNNs)

Team Members:

1. K.D.N.Chandrasena - E/22/054
2. [Member 2 Name] - [Registration No.]
3. [Member 3 Name] - [Registration No.]

September 6, 2026

Abstract

As part of the PeraMorphIQ initiative, this project aims to develop a robust software pipeline for an autonomous DOOM gameplay simulation using Spiking Neural Networks (SNNs). Before hardware deployment on an FPGA-based neuromorphic accelerator, this software phase is critical to verify the model's viability. Leveraging the VizDoom environment and the `snnTorch` framework, our team of three will design, train, and evaluate a Python-based SNN capable of processing visual stimuli and executing in-game actions without human intervention, within a strict 2-month timeline.

1 Introduction

The transition from traditional Artificial Neural Networks (ANNs) to Spiking Neural Networks (SNNs) marks a significant leap towards biologically plausible and energy-efficient artificial intelligence. SNNs transmit information via discrete spikes over time, enabling highly sparse computations ideal for neuromorphic hardware.

The primary goal of this project under PeraMorphIQ is to design a software-based SNN that can autonomously play the classic first-person shooter game, DOOM. By processing continuous visual frames and converting them into discrete spike trains, the SNN will be trained using Surrogate Gradient methods to take optimal actions. This verified Python codebase will serve as the foundational blueprint for the subsequent FPGA deployment phase.

2 Project Objectives

- **Environment Integration:** Establish a seamless bridge between the DOOM game engine and Python using the VizDoom library.
- **Spike Encoding:** Implement real-time Rate/Latency coding mechanisms to translate continuous RGB pixel data into discrete spike trains.
- **SNN Architecture Design:** Develop a Spiking Convolutional Neural Network (SCNN) utilizing Leaky Integrate-and-Fire (LIF) neuron models.
- **Autonomous Training:** Utilize Reinforcement Learning combined with Surrogate Gradient Descent to train the network to navigate, identify enemies, and survive without human input.
- **Hardware Readiness:** Ensure the final Python codebase is structurally modular to facilitate smooth translation to the FPGA neuromorphic accelerator.

3 Methodology

Our technical approach is divided into four main phases:

3.1 Phase 1: Curriculum-Based Training Strategy

Training an SNN from scratch in a complex 3D environment is computationally difficult. Therefore, we will employ a *Curriculum Learning* approach, breaking the DOOM training process into manageable, progressive stages:

- **Stage 1 (Basic Targeting):** Using *basic.cfg*, the SNN will learn to process visual stimuli and perform simple left/right aiming and shooting.
- **Stage 2 (Navigation & Tactics):** Using *deadly_corridor.cfg*, the network will learn to move forward while avoiding hazards and engaging moving targets.
- **Stage 3 (Full Survival):** Using *defend_the_center.cfg*, the SNN will be tested on 360-degree awareness and long-term survival against multiple enemies.

3.2 Phase 2: Spike Encoding & Preprocessing

Extracted frames will be down-sampled and grayscaled to reduce computational overhead. The pixel intensities will be converted into temporal spike trains over T time steps using rate coding techniques provided by `snnTorch`.

3.3 Phase 3: SNN Architecture Formulation

The network will consist of Spiking Convolutional layers for spatial feature extraction, followed by dense LIF layers for action classification. The membrane potential $U(t)$ will be governed by:

$$U[t] = \beta \cdot U[t - 1] + W \cdot X[t] - S[t - 1] \cdot U_{thr}$$

where β is the decay rate and U_{thr} is the firing threshold.

3.4 Phase 4: Training via Surrogate Gradients

Due to the non-differentiable nature of the Heaviside step function in SNNs, standard backpropagation fails. We will implement the *Surrogate Gradient* method (e.g., Fast Sigmoid approximation) during the backward pass to calculate pseudo-gradients, updating network weights via standard PyTorch optimizers.

4 Project Timeline (2 Months)

The project is structured over an 8-week period. Recognizing that environment setup is rapid using VizDoom, the majority of the timeline is heavily dedicated to the computationally intensive SNN training and optimization phases.

Duration	Milestones
Week 1	Environment setup (VizDoom), implementation of spike encoding schemes, and basic SNN architecture design.
Week 2 – 4	Stage 1 Training: Initiating Reinforcement Learning via Surrogate Gradients on basic scenarios. Debugging the reward signals and gradient flow.
Week 5 – 7	Stage 2 & 3 Training: Curriculum progression to complex survival maps. Extensive hyperparameter tuning to achieve autonomous game-play.
Week 8	Final evaluation.

5 Team Structure & Responsibilities

To maximize efficiency, the workload is distributed among the three members:

- **Member 1 (Data & Environment Lead):** Responsible for VizDoom integration, environment configuration, and spike encoding implementation.
- **Member 2 (Architecture & SNN Lead):** Responsible for building the SNN architecture (SCNN + Dense layers) and managing LIF neuron dynamics.
- **Member 3 (Training & Optimization Lead):** Responsible for the Reinforcement Learning loop, surrogate gradient implementation, and hyperparameter tuning.